

ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORES

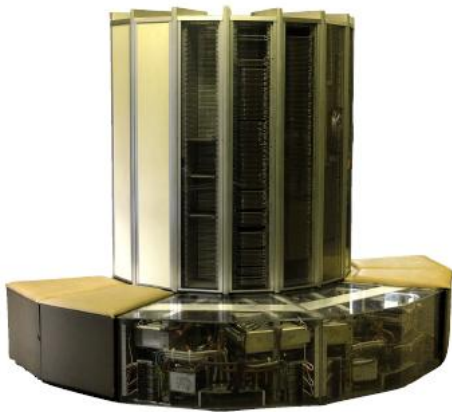
3^{er} curso – Grupo 3

Departamento de Ingeniería y
Tecnología de Computadores

Universidad de Murcia

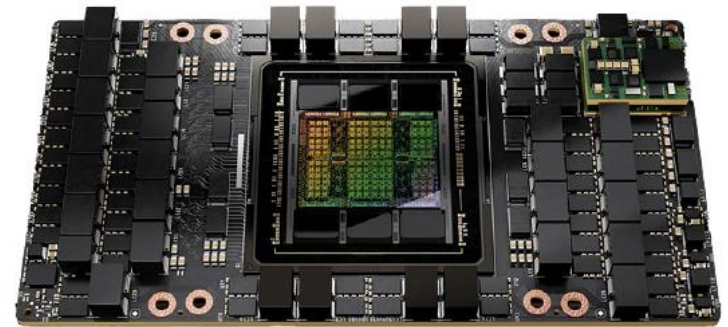
El hardware es sorprendente hoy en día ...

1975



700 Watts / **MegaFLOP**

2024



700 Watts / **PetaFLOP**
1,000,000,000×

Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

The motivating workload

Training a 1B-parameter LLaMA on 30B tokens at 6 FLOPS/param (forward + backward) is roughly $1B \times 30B \times 6 \approx 180$ ExaFLOPS of work. On a small DGX node (~8 GPUs), that completes in about a day. See **FLOPS** for the arithmetic anchor.

The Cray-1 vs Apple M2 contrast

	Cray-1 (1975)	Apple M2 Air (2022)
Peak	160 MFLOPS	~20 TFLOPS (~125,000x)
Memory	8 MB	16 GB
Power	115 kW	a few W
Price	7.9M(56M today)	~\$1500

A Cray-1 running the LLaMA workload above takes ~35,650 years and uses ~1 billion times more energy than the M2.

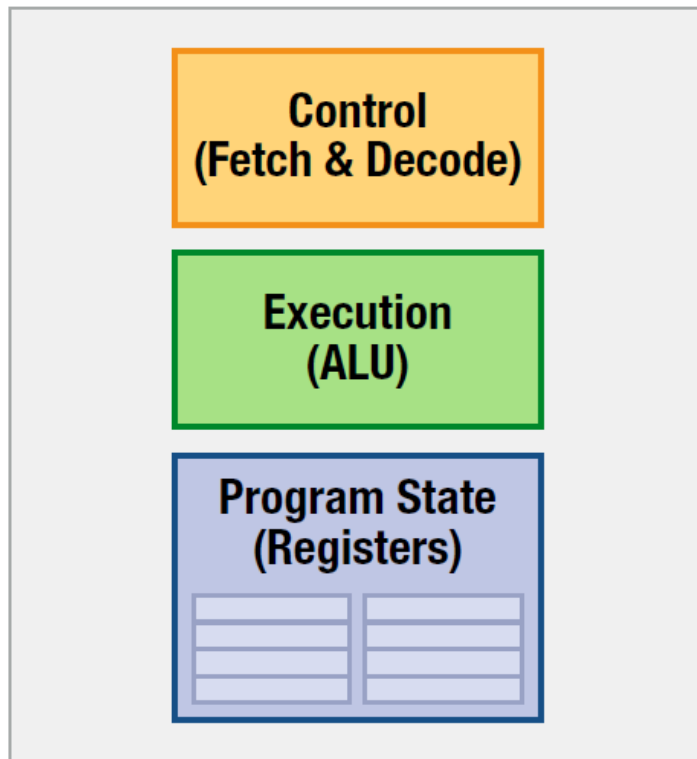
El hardware es sorprendente hoy en día ...

What is a **processor**?
a **programmable** computer
that runs a **sequence of**
instructions over time
including control flow,
computation & state updates

Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

A simple processor an **interpreter** for instructions!



Processor

```
ld    r3, mem[r0+r2]
```

```
ld    r4, mem[r1+r2]
```

```
mul   r3, r3, r4
```

```
add   r5, r5, r3
```

```
addi  r2, r2, 4
```

```
blt   r2, $400, LOOP
```

```
st     addr[r6], r5
```

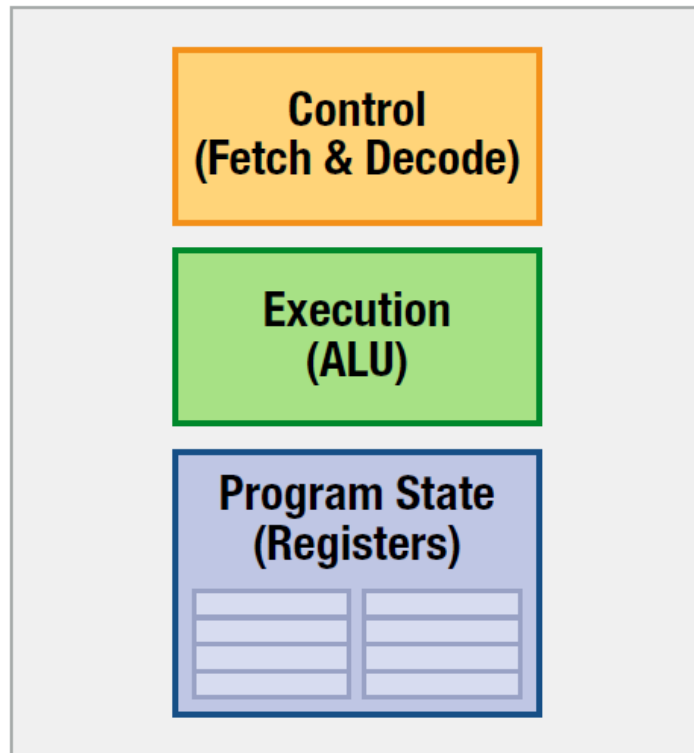
```
...
```

Program

Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

Optimization 1: Increase the **clock speed**



Processor

Higher **voltage**

↳ power grows with v^2

Faster **transistors**

↳ higher leakage

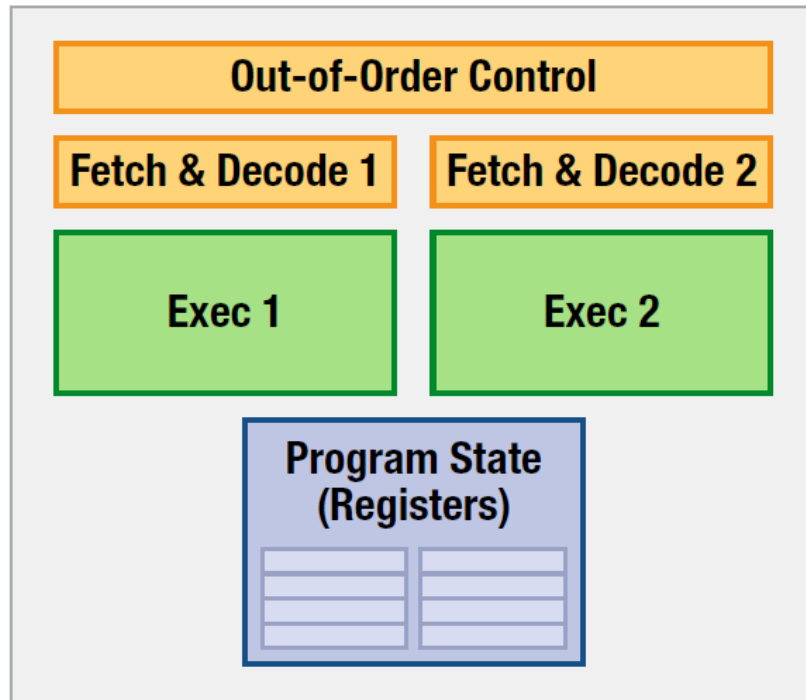
Deeper **pipelining**

↳ lower average IPC

Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

Optimization 2: Execute **multiple** instructions per cycle (**superscalar**)



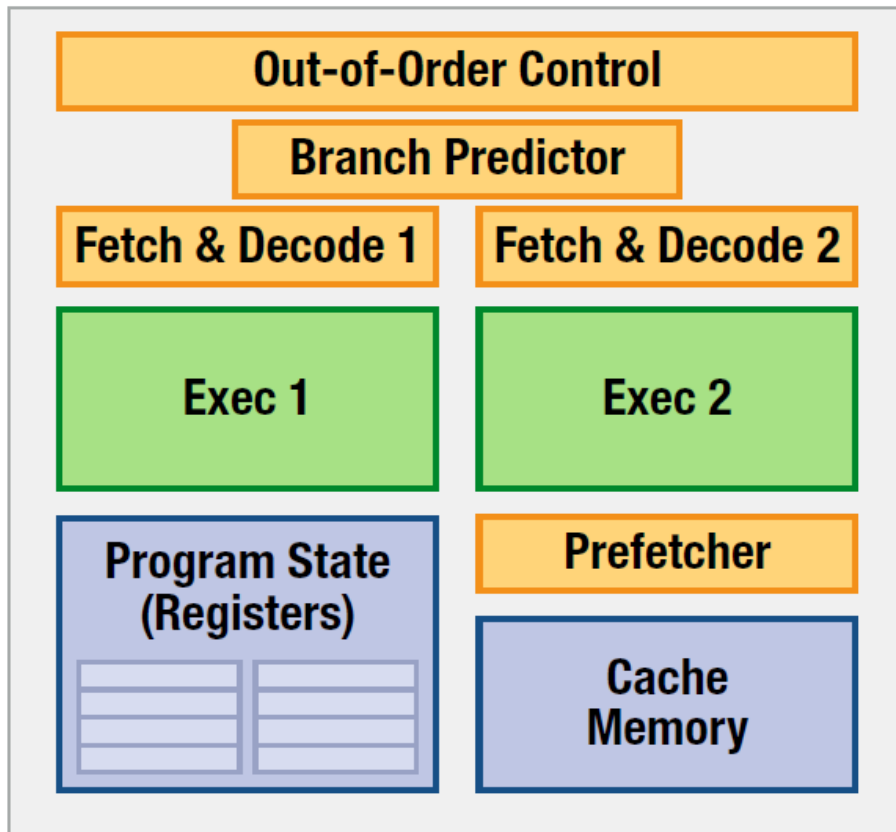
Processor

```
ld    r3, mem[r0+r2]
ld    r4, mem[r1+r2]
mul   r3, r3, r4
add   r5, r5, r3
addi  r2, r2, 4
blt   r2, $400, LOOP
st    addr[r6], r5
...
```

Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

Optimization 3: Avoid stalls through speculation



Processor

```
ld    r3, mem[r0+r2]
```

```
ld    r4, mem[r1+r2]
```

```
mul   r3, r3, r4
```

```
add   r5, r5, r3
```

```
addi  r2, r2, 4
```

```
blt   r2, $400, LOOP
```

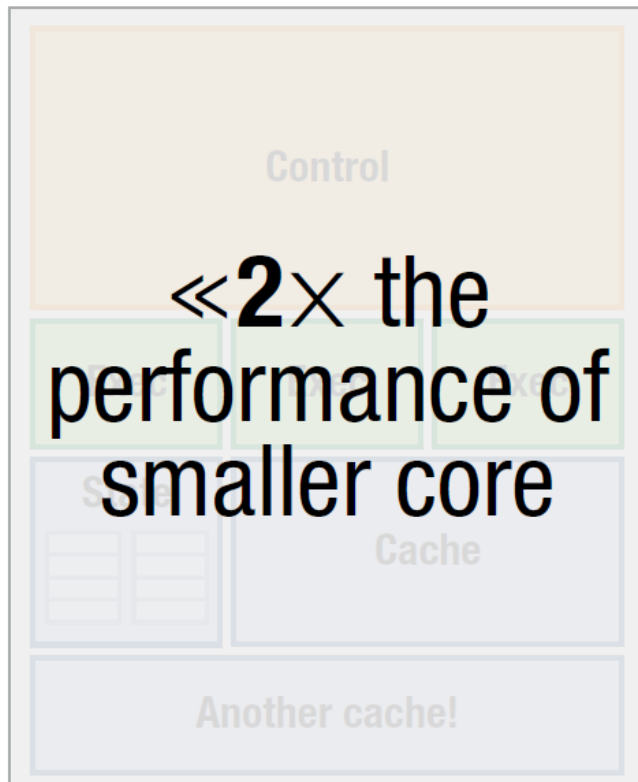
```
st    addr[r6], r5
```

```
...
```

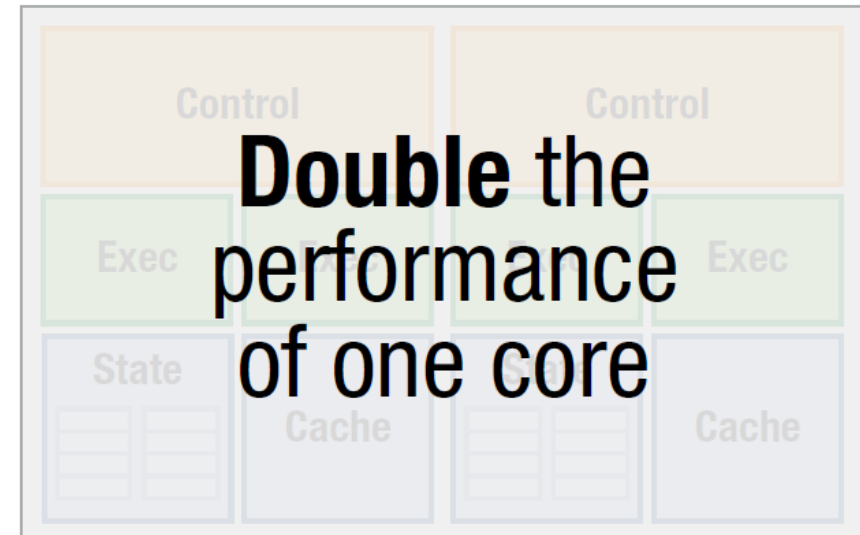
Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

What to do with twice the silicon?



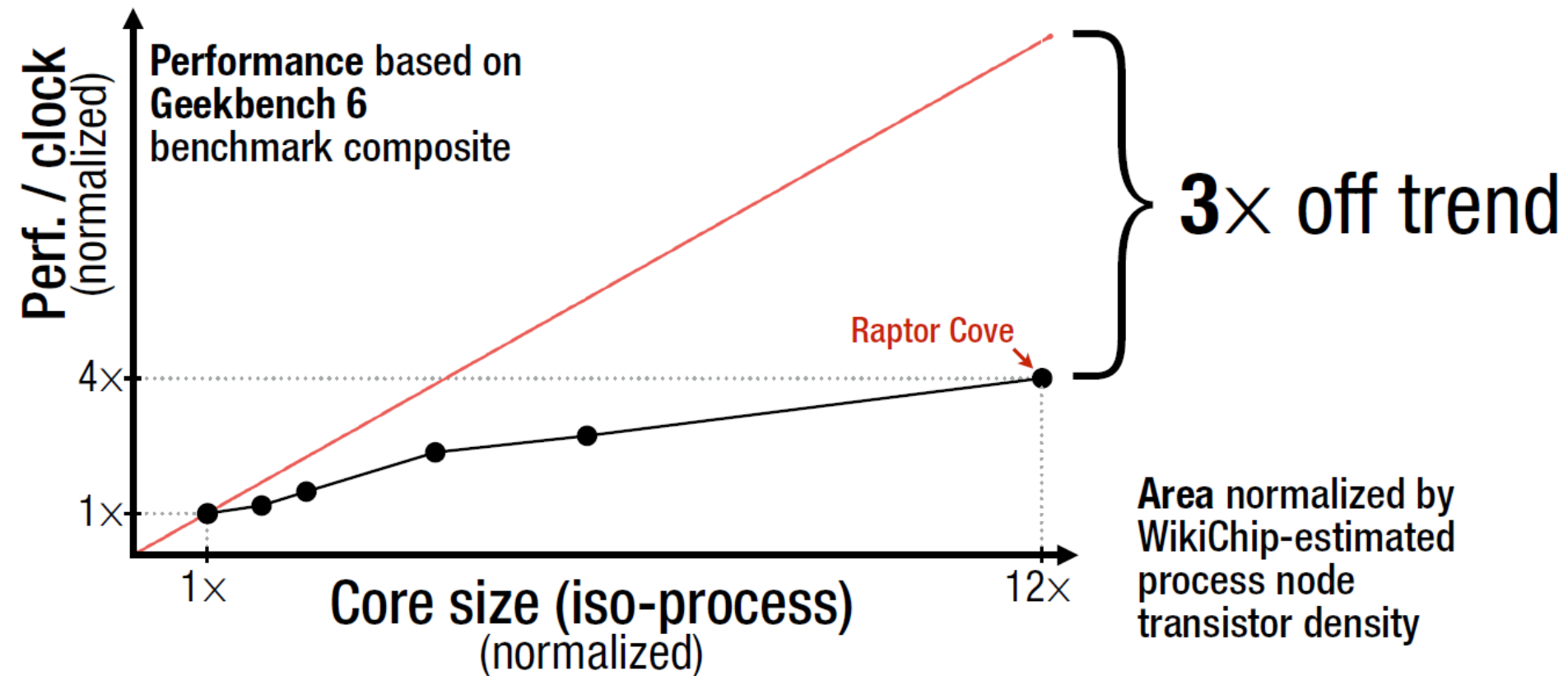
=
equal
area



Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

Diminishing returns to scaling single-core performance



Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

Python

```
# A,B,C : List[float]
# N = 2048
for i in range(N):
    for j in range(N):
        for k in range(N):
            C[i*N + j] += A[i*N + k] * B[k*N + j]
```

10 MegaFLOPS
0.00005% of SoC peak

Apple M2



~20 TeraFLOPs

El hardware es sorprendente hoy en día ...

Version	% Peak
NPU	80%
GPU	15%
CPU Matrix	7.5%
Opt. Par. C	1%
Opt. C	0.2%
C	0.005%
JavaScript	0.0025%
Python	0.00005%

**6 orders of
magnitude
performance
& efficiency**

**Extreme
programming
effort &
specialization
of code**

Apple M2



~20 Tera(FL)OPs

Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

Hardware is amazing, **but...**

the **gap** between
reasonable code
and **peak performance**
is enormous.

clean,
modular,
portable...

To **unlock** the **potential**
of **hardware**,
we need to **rethink how**
we program.

 **1** Better **understand**
hardware

Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

If we want to optimize **throughput**,
is there a **better way** to **scale**
performance?

A **throughput-oriented processor**
exploits abundant parallelism for efficiency

- 1 Scale performance with **multicore**,
not instruction-level parallelism
- 2 Amortize **control overhead**
with **SIMD execution**
- 3 Hide **latency** with **concurrent**
threads, not speculation

Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

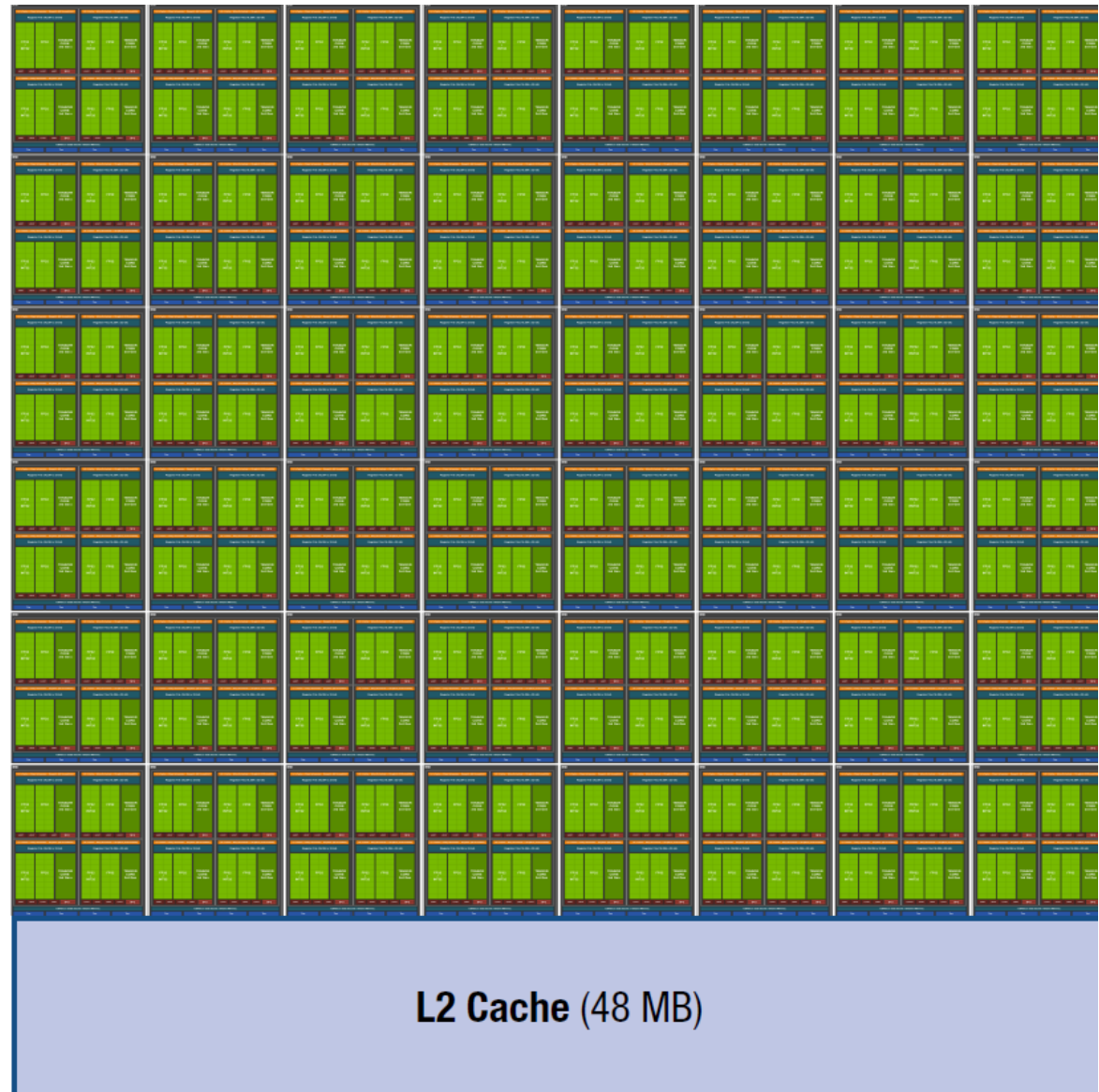
Our whole GPU

48 SMs
 x 4 = 192 cores
 x32 = 6144 exec. units
 @ 2.175 GHz
 = 26.7 TFLOPS

192x12 = 2,304 threads

96k 1024-bit registers
 = 12 MB

48 MB L2 cache



Slides inspired by Jonathan Ragan-Kelley (MIT)

El hardware es sorprendente hoy en día ...

- 1 **Better understand hardware**
- 2 **Practice programming it for high performance**
- 3 Reasoning about performance from **first principles**

Slides inspired by Jonathan Ragan-Kelley (MIT)

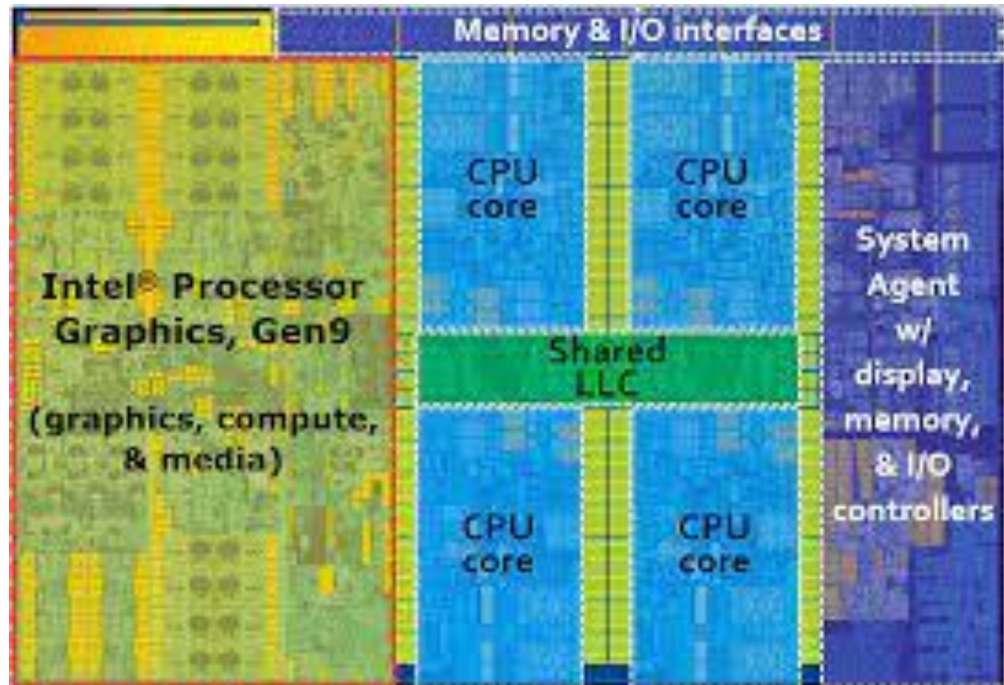
Contexto: evolución de las arquitecturas (I)

- Gran avance en las máquinas desde el primer computador electrónico de propósito general (60 años)
 - Un ordenador de hoy de menos de 500 € ofrece más rendimiento, memoria y disco que un supercomputador de los años 80 de más de 1M €
- Los motivos de esta importante mejora han sido:
 - Avances en tecnología (Ley de Moore)
 - En 1965 Gordon E. Moore (cofundador de Intel) notó que el número de componentes en los circuitos integrados se había doblado cada dos años desde su invención en 1958, y predijo que esta tendencia continuaría durante “al menos 10 años”.
 - Avances en el diseño de la arquitectura de los computadores (segmentación, paralelismo, principio de localidad)

Contexto: evolución de las arquitecturas (II)

- Procesadores multinúcleo (CMP)
- Hoy en día encontramos dispositivos multiprocesador en todos los segmentos de los ordenadores:

Todo en el mismo chip (CMP)

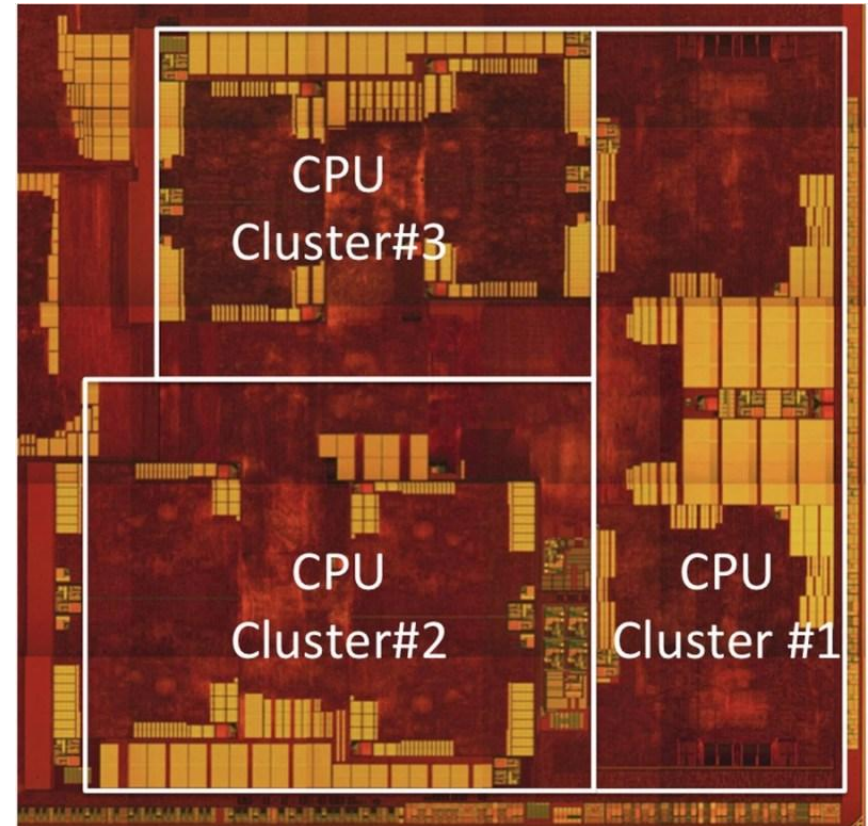


**Intel Core i7-6700K
Skylake**

Contexto: evolución de las arquitecturas (III)

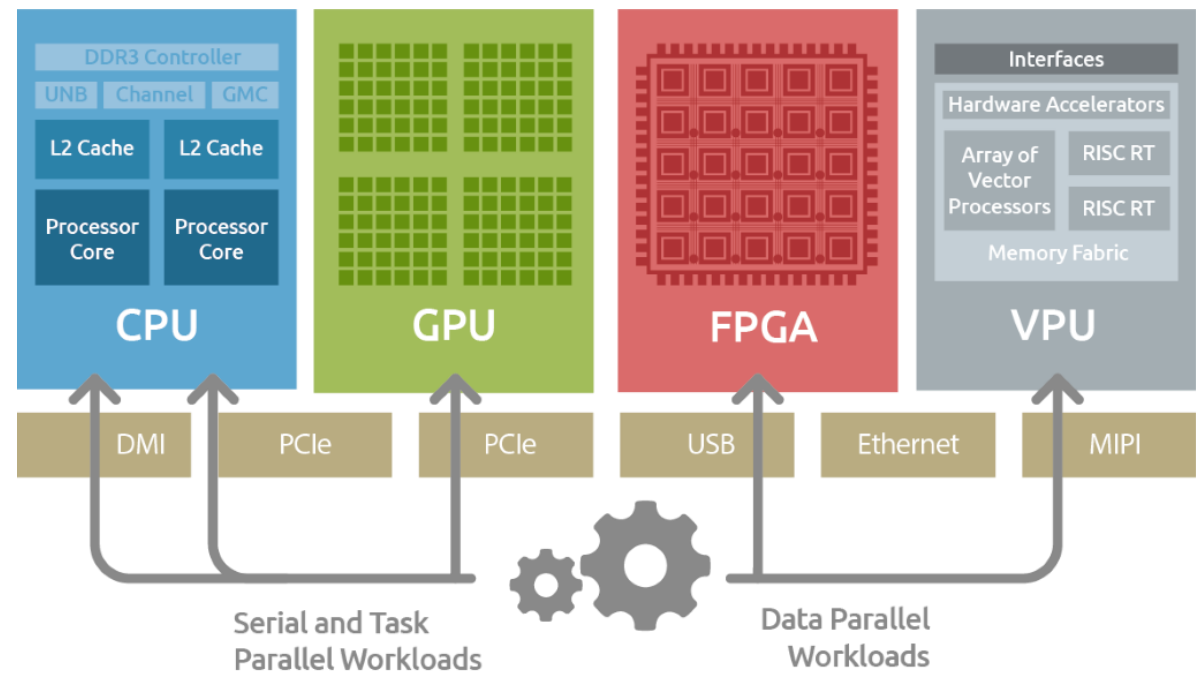
- Arquitecturas heterogéneas
- **Multinúcleo heterogéneo:** Todos los núcleos no tiene que ser idénticos.

Die shot del Mediatek Helio X20 (MT6797) con un cluster de cuatro núcleos Cortex-53 ULP de **muy bajo consumo (#1)**, otro cluster Cortex-A53 de cuatro núcleos **eficiente (#2)**, y un tercer cluster de **alto rendimiento** de dos núcleos Cortex-A72 (**#3**) –
Fuente: Wikichip



Contexto: evolución de las arquitecturas (IV)

- Arquitecturas con aceleradores
- La arquitectura con aceleradores se refiere a un entorno con procesadores y aceleradores especializados ([CPUs](#), [GPU](#), TPUs, FPGAs, ...) que trabajan juntos para realizar diferentes tareas. El objetivo es optimizar el rendimiento, la eficiencia energética y la rentabilidad.

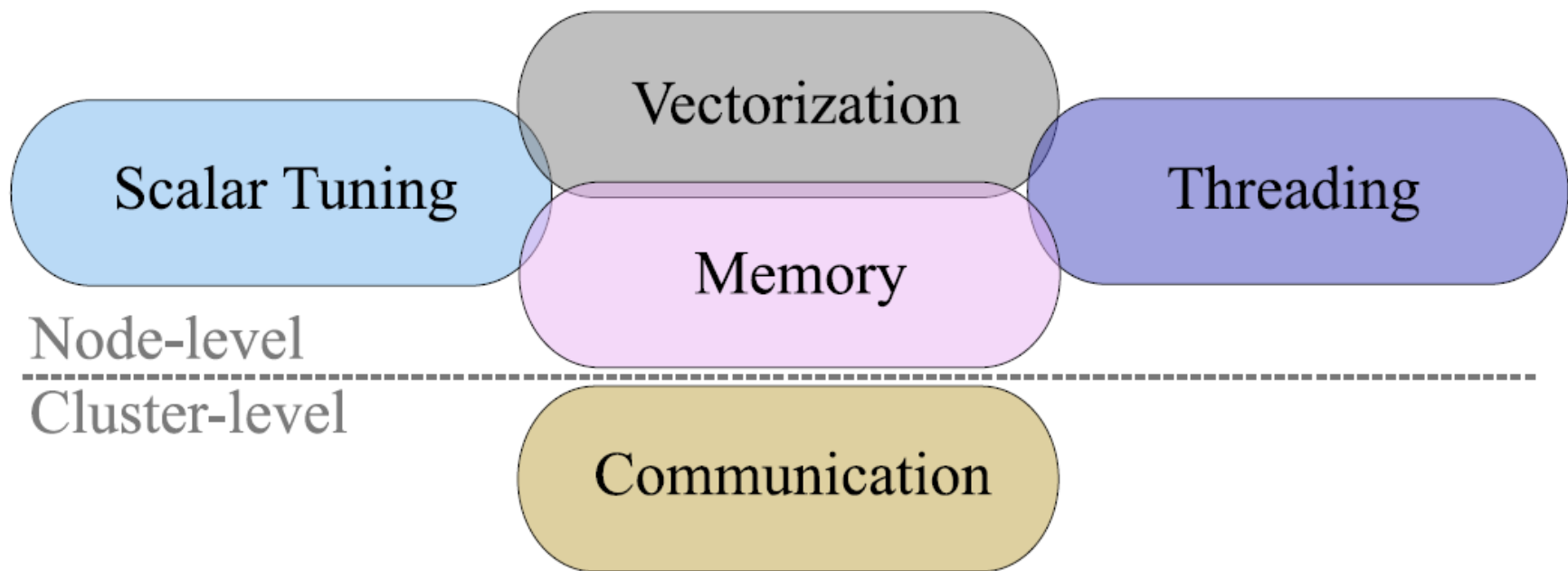


Arquitectura heterogénea con aceleradores para IA

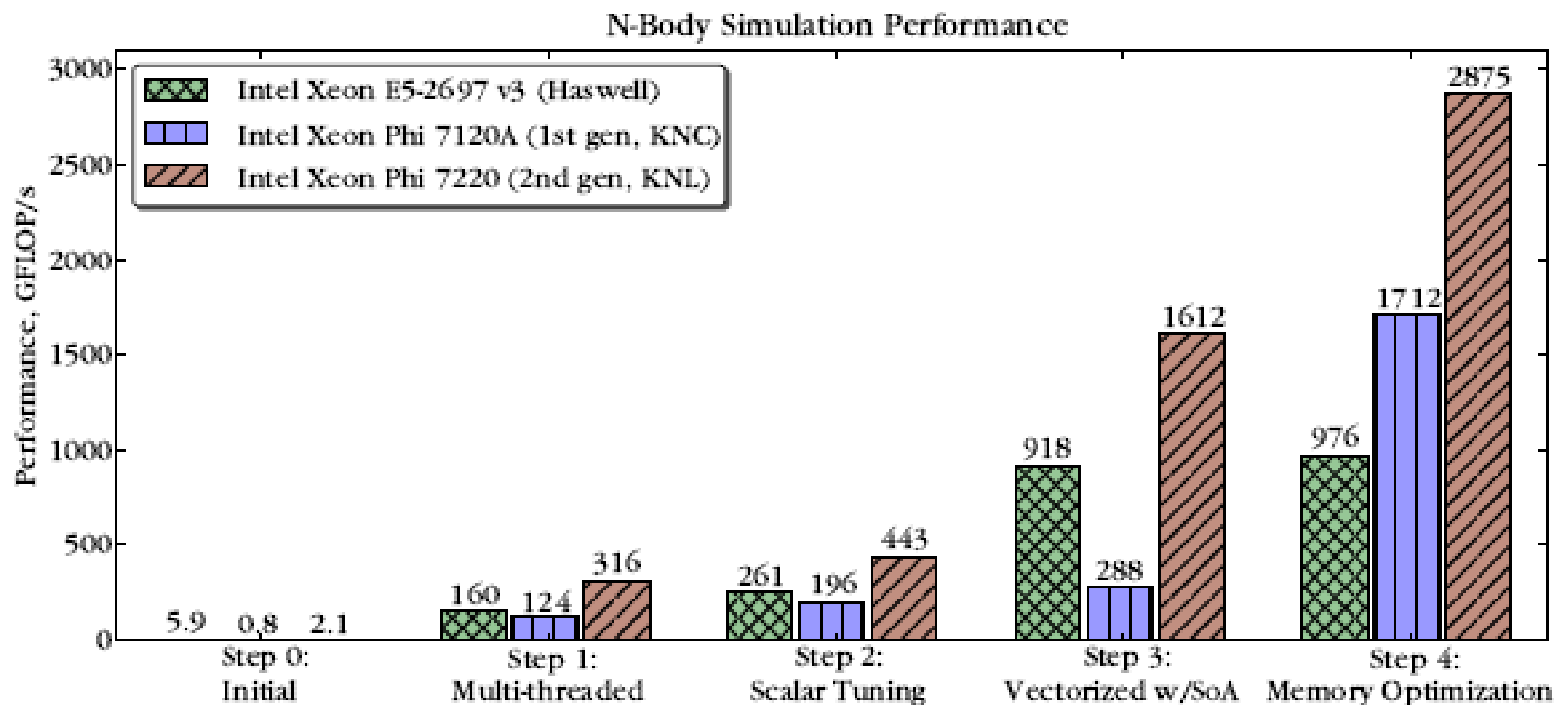
Evolución de las arquitecturas: resumen

- Monociclo: Simplicidad, pero gran ineficiencia
- Multiciclo: ¿cómo mejora al monociclo?
- Segmentación
 - Ejecución **solapada** de varias instrucciones
 - Detenciones debido a los riesgos y **dependencias**
- **Explotando el ILP**
 - Superescalares, Ejecución fuera de orden, Especulación
 - MultiThreading, ...
- **CMPs (multiprocesadores en un chip)**
 - Vectorización
 - Paralelismo entre núcleos (cores)
- **Arquitecturas Heterogéneas**
 - Multinúcleo heterogéneo
 - Uso de aceleradores: GPUs, DSPs, TPUs, XPU, ...
- Supercomputadores

Optimización de código



Optimización de código



Presentación (I)

- Curso académico: 2026-2027
- Curso: 3º
- Créditos: 6 ECTS
- Estimación del volumen de trabajo del alumno: 150 hrs.
- Tipo de Enseñanza: Docencia presencial
- Clases Teoría: 2 hrs semanales
 - Aula B.01
 - Horario: jueves 15:30 – 17:30

Presentación (II)

- Prácticas:
 - Subgrupo 3.1: lunes 17:10 - 18:50: Lab 1.7
 - Subgrupo 3.2: martes 15:30 - 17:10: Lab 1.5
 - Subgrupo 3.3: lunes 15:30 - 17:10: Lab 1.6

- Prácticas:
 - P1 (3 sesiones): semana 14/sep
 - P2 (4 sesiones): semana 12/oct
 - P3 (4 sesiones): semana 9/nov

- El examen de prácticas de la Convocatoria I será la última semana (la del 30/nov) en el horario de cada grupo.

Descripción

- La asignatura tiene como objetivo general el mostrar a los alumnos los **diferentes modelos de arquitectura** existentes y que hacen uso del **paralelismo** a distintos niveles para aumentar el rendimiento.
- La asignatura se centra en el modelo de **multiprocesador en un único chip** (single-chip multiprocessor) utilizado por la mayoría de los fabricantes de microprocesadores para cualquier diseño de microprocesador en cualquiera de sus ámbitos de utilización posibles.
- Además, la asignatura da una introducción a las **arquitecturas heterogéneas**, ya sean con **procesadores de distinto tipo** en un único chip, o **procesadores con aceleradores**.
- Se trata de conseguir que el alumno comprenda los principales aspectos de diseño de este tipo de arquitecturas, tanto desde un punto de vista **cualitativo** como **cuantitativo**, para así poder analizar y **evaluar** las diferentes alternativas de diseño en los sistemas actuales.

Descripción

- La asignatura aborda:
 1. El estudio de los aspectos relacionados con la arquitectura de cada uno de los núcleos de ejecución, los cuales explotan el paralelismo a nivel de instrucción (**ILP**) así como fundamentalmente el paralelismo de datos (vectorización o ejecución **SIMD**) para mejorar su rendimiento.
 2. Nos centramos en la problemática de usar de manera eficiente los diversos **núcleos** que hay en un chip, más concretamente la organización de la jerarquía de memoria y la sincronización y comunicación entre los mismos, teniendo en cuenta la red de interconexión.
 3. Todo el desarrollo realizado se complementa con el estudio de ejemplos de **sistemas actuales**.

Temario Teoría

Presentación de la asignatura

Tema 1. Arquitectura de un multiprocesador-en-un-chip

- 1.1 Aspectos tecnológicos: Leyes de Moore y de Dennard
- 1.2 Clasificación de Flynn. Principales figuras de mérito
- 1.3 Aspectos paralelos: Leyes de Amdahl y Gustavson
- 1.4 De sistemas empujados a supercomputadores

Tema 2. Explotando el paralelismo dentro de cada núcleo

- 2.1 Procesadores de emisión múltiple: núcleos superescalares
- 2.2 ILP, planificación dinámica y especulación
- 2.3 Otros diseños: multihilo simultáneo (SMT)
- 2.4 La vectorización e instrucciones SIMD
- 2.5 Ejemplos de arquitecturas modernas

Temario Teoría

Tema 3. Explotando el paralelismo entre los núcleos

- 3.1 La coherencia de caches
- 3.2 Protocolos de fisgoneo y de directorio
- 3.3 Sincronización entre hilos
- 3.4 Modelos de consistencia de memoria
- 3.5 Redes de interconexión on-chip

Tema 4. Arquitecturas heterogéneas

- 4.1 Arquitecturas heterogéneas a nivel de CPU
- 4.2 Arquitecturas heterogéneas usando aceleradores (GPUs, FPGAs, TPUs, etc)

Programa de Prácticas

- **Práctica 1.** Introducción a la computación de alto rendimiento
- **Práctica 2.** Procesadores CMP: usando la vectorización y la paralelización
- **Práctica 3.** Optimización del rendimiento de un procesador CMP

Las prácticas se realizan de forma INDIVIDUAL

Competencias

1. Adquirir una visión general de la **arquitectura de una máquina paralela**, prestando especial atención a las arquitecturas multinúcleo actuales.
2. Conocer *distintas alternativas actuales y futuras en el diseño de las máquinas paralelas, siendo capaz de discutir sus ventajas/inconvenientes.*
3. Ser capaz de **identificar** aquellas partes de un sistema que son susceptibles de mejora e **investigar** diversas alternativas.
4. Saber cómo evaluar las prestaciones de una arquitectura paralela en función de la carga de trabajo prevista para la misma.
5. Entender la relación entre el **modelo de programación** de una máquina paralela y la **arquitectura** de la misma.
6. Estudiar el funcionamiento y principales parámetros de diseño de los **procesadores superescalares**.

Competencias

7. Conocer y comprender el problema de la **coherencia** de las caches y algunas soluciones sencillas al mismo.
8. Entender diversas implementaciones básicas de los mecanismos de **sincronización** necesarios en estas arquitecturas.
9. Conocer algunos de los parámetros básicos de diseño de la **red de interconexión** de una arquitectura paralela.

Resultados de aprendizaje

Saber responder a preguntas como:

- ¿Cuánto tarda en ejecutarse una aplicación de 10^{12} instrucciones?
- ¿Cómo puedo hacer que mi programa aproveche los recursos de mi procesador para ir más rápido?
- ¿Por qué el tamaño de una línea de cache es el que es?
- ¿Cómo se comunican los hilos entre sí?
- ¿Cómo funcionan *Meltdown* y *Spectre*?

METODOLOGÍA DOCENTE

- Clases de teoría
 - Las transparencias de los temas en [Recursos](#) del AV.
- Prácticas de laboratorio
 - En el laboratorio correspondiente a cada grupo.
 - El enunciado y el material de cada práctica en [Recursos](#) del AV
 - No hay que entregar nada.
- Tutorías (por email y por [Videoconferencia](#) del AV)

Evaluación (I)

- Examen final de teoría (70%):
 - Examen teórico-práctico: constará de preguntas y problemas de teoría.
 - Día de enero de 2027 (mañana)
 - Día de mayo de 2027 (tarde)
 - Día de junio de 2027 (tarde)
- Prácticas (30%):
 - Convocatoria I: Examen práctico delante del ordenador, con preguntas relativas a las 3 prácticas. Dicho examen práctico se realizará en la última sesión de la última práctica.
 - Convocatorias II y III: En la fecha del examen de teoría habrá un apartado de preguntas teóricas sobre las 3 prácticas, que determinarán la nota de la parte de prácticas.

Evaluación (II)

- **Aplicable a las tres convocatorias:** Para aprobar la asignatura la nota media total de la asignatura debe de ser igual o mayor que 5.
- Para poder hacer media con las prácticas, habrá de obtenerse en el examen teórico una calificación **mayor o igual a 4** (sobre 10). Una vez superada la parte teórica de la asignatura, la nota se conservará hasta la última convocatoria del curso académico.
- Para poder hacer media con el examen de teoría, habrá de obtenerse en las prácticas una calificación **mayor o igual a 4** (sobre 10). La nota de la parte práctica se conservará hasta la última convocatoria del curso académico.
- Obtendrá la calificación de "No Presentado" el alumno que no haya realizado el examen teórico de la convocatoria.

Profesorado y recursos

- José Manuel García Carrasco

- Profesor Teoría
- Correo electrónico: jmgarcia@um.es
- Página web: <http://webs.um.es/jmgarcia>
- Horario de tutorías:
 - Jueves de 10 a 13 hrs
 - Tutorías electrónicas: A través del email/Aula Virtual
- Despacho 3.44, Facultad de Informática, 3ª planta

- Alexandra Jimborean

- Profesora Prácticas 3.1 y 3.3
- Correo electrónico: alexandra.jimborean@um.es
- Horario de tutorías:
 - Tutorías electrónicas: A través del email/Aula Virtual

- Juan Luis Aragón Alcaraz

- Profesor Prácticas 3.2
- Correo electrónico: jlaracon@um.es
- Horario de tutorías:
 - Tutorías electrónicas: A través del email/Aula Virtual

- Recursos:

- Todo el material de la asignatura estará disponible a través del Aula Virtual de la UMU (<http://aulavirtual.um.es>).
- De Andrej Karpathy (uno de los fundadores de OpenAI)
 - Consejos para [undergrads](#) y para aquellos que [considering or pursuing a PhD](#)
- Videos motivacionales
 - Steve Jobs: <https://www.youtube.com/watch?v=Tuw8hxrFBH8>
 - Roger Federer: <https://www.youtube.com/watch?v=b51GMoYyGmQ>

Bibliografía básica

- J.L. Hennessy y D.A. Patterson. «Computer Architecture: A Quantitative Approach». 6a edición, Morgan Kaufmann Publishers, Inc., 2018.
- J. Ortega, M. Anguita y A. Prieto. «Arquitectura de Computadores». Thompson, 2005.
- A. S. Tanenbaum. «Organización de Computadoras. Un enfoque estructurado». Prentice-Hall, 4a Edición en español, 2000.
- D. Culler. «Parallel Computer Architecture. A Hardware/Software Approach». Ed. Morgan Kaufmann Publishers, 1998.
- D. A. Patterson and J. L. Hennessy. «Computer Organization and Design: The Hardware/Software Interface, 5th Edition, Revised Printing». Morgan Kaufmann Publishing Co., 2013.
- D.H. Bailey, R.F. Lucas and S. Williams. «Performance Tuning of Scientific Applications». CRC Press. 2010.

Bibliografía detallada por temas

Tema 1

- Capítulo 7 de [Ortega, Anguita y Prieto 2004]
- Capítulo 1 de [Hennessy y Patterson 2019, Sixth Edition]
- Capítulo 1 de [Culler 1998]

Tema 2

- Capítulos 3 y 4 de [Ortega, Anguita y Prieto 2004]
- Capítulo 3 de [Hennessy y Patterson 2019, Sixth Edition]

Tema 3

- Capítulo 10 de [Ortega, Anguita y Prieto 2004]
- Capítulo 4 de [Hennessy y Patterson 2019, Sixth Edition]
- Capítulo 5, 7 y 8 de [Culler 1998]

Tema 4

- Capítulo 7 de [Hennessy y Patterson 2019, Sixth Edition]

Posibles conclusiones al acabar el curso

- "¡Las arquitecturas de los ordenadores son fascinantes, y realmente quiero entender por qué los programas aparentemente simples pueden comportarse de formas tan complejas!"
- "Quiero aprender a diseñar algoritmos que funcionen muy rápido, no importa lo complicado que sea la arquitectura subyacente de la computadora".
- "¡Espero que la mayor parte del tiempo pueda utilizar el software eficiente (y rápido) que otros hayan escrito, para que no tenga que preocuparme por los detalles internos de la arquitectura!"
- Todo lo anterior, en diferentes momentos.

Créditos

- Slides from 263-2800 "Design of Parallel and High-Performance Computing" Profs. Torsten Hoefler and Markus Püschel, ETHZ Zurich, 2016, htor@inf.ethz.ch
- Slides from CS267 "Applications of Parallel Computers". Prof. Jim Demmel, EECS & Math Departments @ Berkeley (USA), 2015. demmel@berkeley.edu
- Slides from 263-2300 "How to Write Fast Numerical Code". Prof. Markus Püschel, ETHZ Zurich, 2016, pueschel@inf.ethz.ch
- Slides from CS252 "Graduate Computer Architecture". Prof. Krste Asanovic, EECS and CS Departments at UC Berkeley (USA), 2015. krste@eecs.berkeley.edu
- Slides from Introduction to High Performance Computing: Parallel Computing, Distributed Computing, Grid Computing and More. Dr. Jay Boisseau, Director, Texas Advanced Computing Center, boisseau@tacc.utexas.edu

ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORES

3^{er} curso – Grupo 2

Departamento de Ingeniería y
Tecnología de Computadores

Universidad de Murcia